KOIOS · THE PLAYGROUND

Simulations: what was built, and how

Thirty-one simulators and the templates to learn on them; a telemetry every run writes and a pane that draws it; a live 3D view through the Companion; experiments filed as they end, with notes; a paper's reimplementation wired to a simulator by the coding agents – and the process that made it: every run, every finding, every review.

Branch claude/zealous-bardeen-vr8z49 at 628e567 · 2026-10-19 · the two documents this prints are docs/simulation.md and docs/simulation-process.md

Part 1 · What was built

- What is built
- The telemetry format (schema 1)
- Challenges, and how they are handled
- Verified
- Next

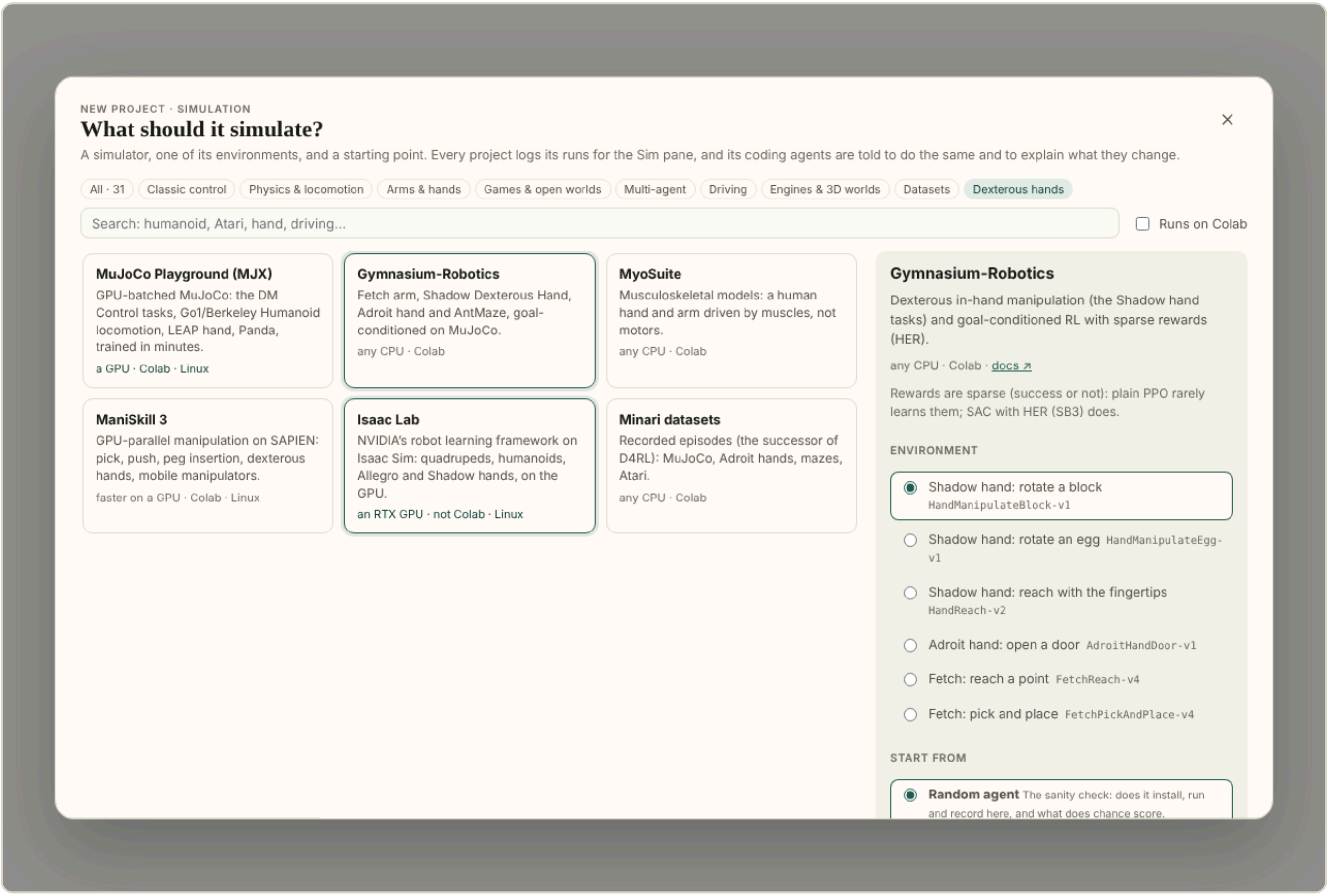
Part 2 · How it was built, checked and reviewed

- 1. The ask
- 2. The order of work
- 3. The background tasks, one by one
- 4. What testing found that reading would not have
- 5. Verification matrix
- 6. The review: does it break anything that was there?
- 6.1 A second look, with a game and a made-up world
- 6.2 Live 3D
- 6.3 Experiments and the paper toolkit
- 7. Checklist: before merging, or after changing any of it
- 8. How to extend it

PART 1

What was built

The Playground can start a **simulation project**: any of 31 simulators, one of its environments, and a starting point (a random agent, PPO, SAC, Stable-Baselines3, a world model from scratch, DreamerV3, TD-MPC2, Isaac Lab). Every project writes its runs in one format, `koios_telemetry`, and the **Sim pane** beside the editor draws them as they happen: episodes and returns, the numbers, the videos, a world model's dreams against what really happened, the notes that the code and the coding agents leave on the timeline, and what looks wrong with what to try. Coding agents (Claude Code, Codex, the page's own) are told to use the telemetry and to explain each change in `LEARNING.md`, so the person can follow and learn.



NEW PROJECT · SIMULATION

What should it simulate?

A simulator, one of its environments, and a starting point. Every project logs its runs for the Sim pane, and its coding agents are told to do the same and to explain what they change.

☐ Runs on Colab

MuJoCo Playground (MJX)
GPU-batched MuJoCo: the DM Control tasks, Go1/Berkeley Humanoid locomotion, LEAP hand, Panda, trained in minutes.
a GPU · Colab · Linux

Gymnasium-Robotics
Fetch arm, Shadow Dexterous Hand, Adroit hand and AntMaze, goal-conditioned on MuJoCo.
any CPU · Colab

MyoSuite
Musculoskeletal models: a human hand and arm driven by muscles, not motors.
any CPU · Colab

ManiSkill 3
GPU-parallel manipulation on SAPIEN: pick, push, peg insertion, dexterous hands, mobile manipulators.
faster on a GPU · Colab · Linux

Isaac Lab
NVIDIA's robot learning framework on Isaac Sim: quadrupeds, humanoids, Allegro and Shadow hands, on the GPU.
an RTX GPU · not Colab · Linux

Minari datasets
Recorded episodes (the successor of D4RL): MuJoCo, Adroit hands, mazes, Atari.
any CPU · Colab

Gymnasium-Robotics

Dexterous in-hand manipulation (the Shadow hand tasks) and goal-conditioned RL with sparse rewards (HER).

any CPU · Colab · docs ↗

Rewards are sparse (success or not): plain PPO rarely learns them; SAC with HER (SB3) does.

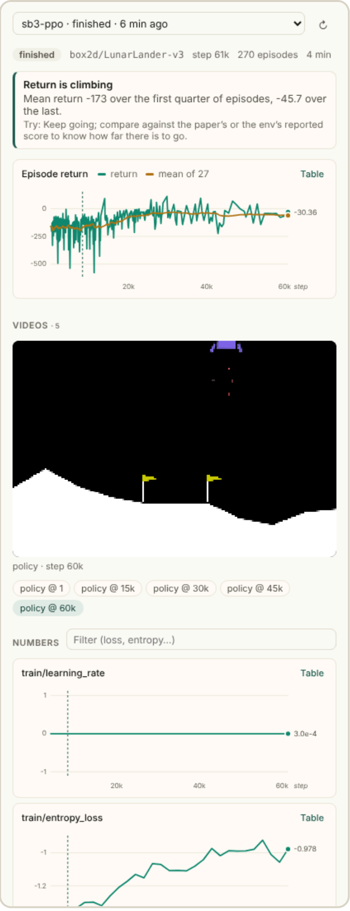
ENVIRONMENT

● Shadow hand: rotate a block `HandManipulateBlock-v1`
○ Shadow hand: rotate an egg `HandManipulateEgg-v1`
○ Shadow hand: reach with the fingertips `HandReach-v2`
○ Adroit hand: open a door `AdroitHandDoor-v1`
○ Fetch: reach a point `FetchReach-v4`
○ Fetch: pick and place `FetchPickAndPlace-v4`

START FROM

● **Random agent** The sanity check: does it install, run and record here, and what does chance score.

the picker on "Dexterous hands": six simulators with hands, Gymnasium-Robotics chosen, its Shadow-hand environments and the templates



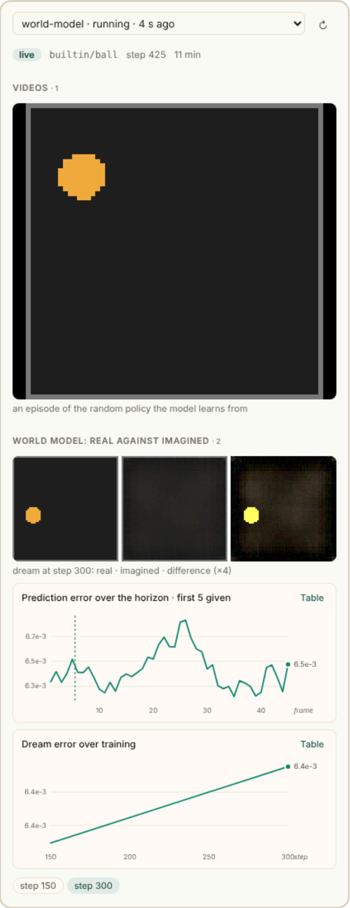
sb3-ppo · finished · 6 min ago

finished box2d/LunarLander-v3 step 61k 270 episodes 4 min

Return is climbing
Mean return -173 over the first quarter of episodes, -45.7 over the last.
Try: Keep going; compare against the paper's or the env's reported score to know how far there is to go.

Episode return
return
mean of 27
Table
0
-250
-500
-30.36
20k
40k
60k
step
VIDEOS · 5
policy · step 60k
policy @ 1
policy @ 15k
policy @ 30k
policy @ 45k
policy @ 60k
NUMBERS
Filter (loss, entropy...)
train/learning_rate
Table
1
0
-1
3.0e-4
20k
40k
60k
step
train/entropy_loss
Table
-1
-1.2
-0.978

A game: SB3's PPO on LunarLander, finished the Sim pane: finished, "Return is climbing" from −173 to −46, the return chart with an agent's note as a dashed mark, five videos of the policy with the lander on screen, the numbers one chart each



world-model · running · 4 s ago
live
builtin/ball
step 425
11 min
VIDEOS · 1

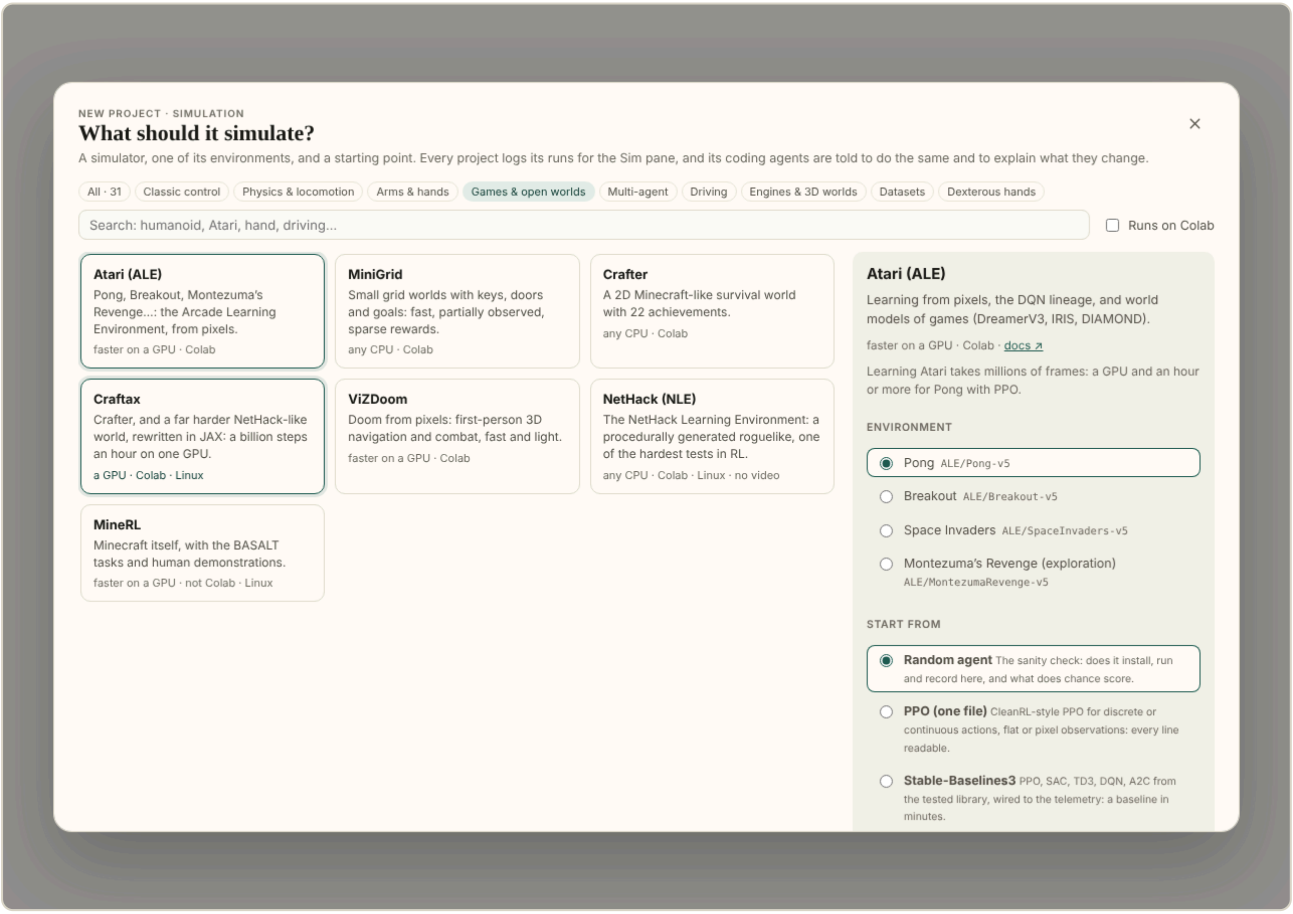
an episode of the random policy the model learns from

WORLD MODEL: REAL AGAINST IMAGINED · 2

dream at step 300: real · imagined · difference (×4)

Prediction error over the horizon · first 5 given
Table
6.7e-3
6.5e-3
6.3e-3
6.5e-3
10
20
30
40
frame
Dream error over training
Table
6.4e-3
6.4e-3
6.4e-3
150
200
250
300step
step 150
step 300

A made world: a world model dreaming the ball world the Sim pane on a world-model run: the data video, the dream as real · imagined · difference, the error over the horizon rising after the 5 given frames, the error over training



NEW PROJECT · SIMULATION

What should it simulate?

A simulator, one of its environments, and a starting point. Every project logs its runs for the Sim pane, and its coding agents are told to do the same and to explain what they change.

Search: humanoid, Atari, hand, driving...

☐ Runs on Colab

Atari (ALE)
Pong, Breakout, Montezuma's Revenge...: the Arcade Learning Environment, from pixels.
faster on a GPU · Colab

MiniGrid
Small grid worlds with keys, doors and goals: fast, partially observed, sparse rewards.
any CPU · Colab

Crafter
A 2D Minecraft-like survival world with 22 achievements.
any CPU · Colab

Craftax
Crafter, and a far harder NetHack-like world, rewritten in JAX: a billion steps an hour on one GPU.
a GPU · Colab · Linux

ViZDoom
Doom from pixels: first-person 3D navigation and combat, fast and light.
faster on a GPU · Colab

NetHack (NLE)
The NetHack Learning Environment: a procedurally generated roguelike, one of the hardest tests in RL.
any CPU · Colab · Linux · no video

MineRL
Minecraft itself, with the BASALT tasks and human demonstrations.
faster on a GPU · not Colab · Linux

Atari (ALE)

Learning from pixels, the DQN lineage, and world models of games (DreamerV3, IRIS, DIAMOND).

faster on a GPU · Colab · docs ↗

Learning Atari takes millions of frames: a GPU and an hour or more for Pong with PPO.

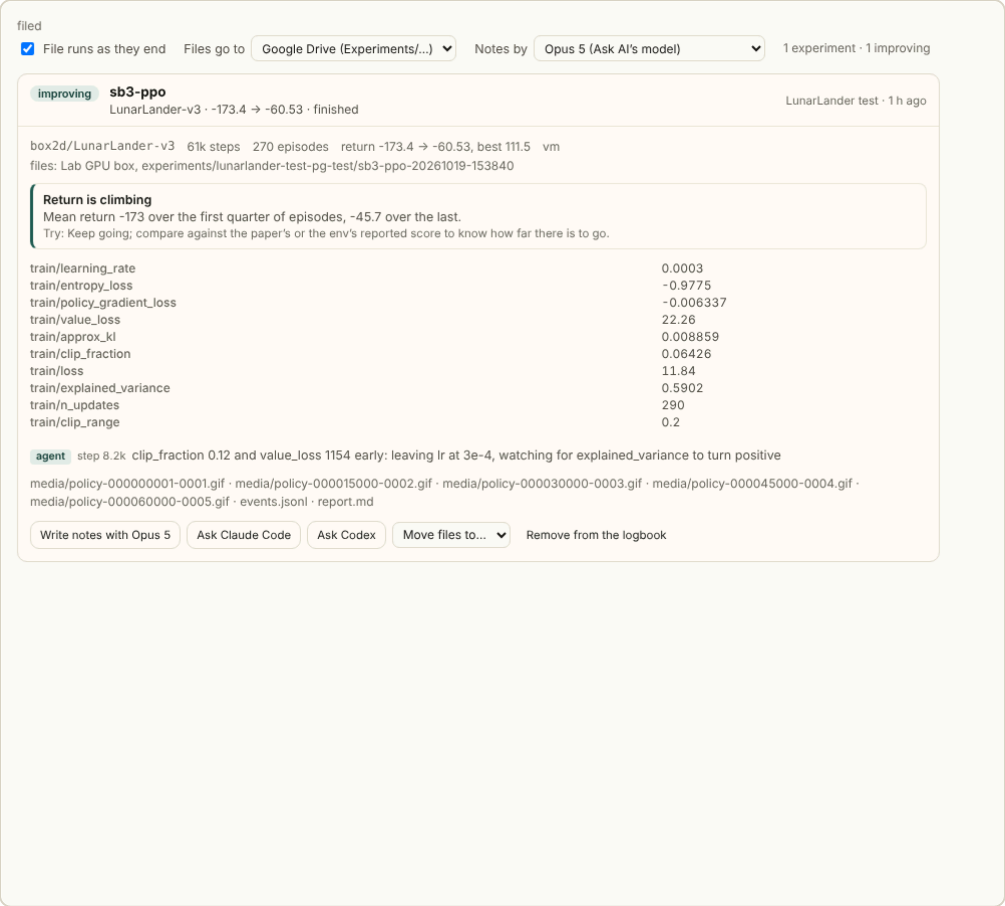
ENVIRONMENT

- ● Pong `ALE/Pong-v5`
- ○ Breakout `ALE/Breakout-v5`
- ○ Space Invaders `ALE/SpaceInvaders-v5`
- ○ Montezuma's Revenge (exploration) `ALE/MontezumaRevenge-v5`

START FROM

- ● **Random agent** The sanity check: does it install, run and record here, and what does chance score.
- ○ **PPO (one file)** CleanRL-style PPO for discrete or continuous actions, flat or pixel observations: every line readable.
- ○ **Stable-Baselines3** PPO, SAC, TD3, DQN, A2C from the tested library, wired to the telemetry: a baseline in minutes.

the picker on "Games & open worlds": Atari, MiniGrid, Crafter, Craftax, ViZDoom, NetHack, MineRL, with Atari's environments and templates



filed

☑ File runs as they end · Files go to: Google Drive (Experiments/...) · Notes by: Opus 5 (Ask AI's model) · 1 experiment · 1 improving

improving **sb3-ppo**

LunarLander-v3 · -173.4 → -60.53 · finished

LunarLander test · 1 h ago

`box2d/LunarLander-v3` 61k steps 270 episodes return -173.4 → -60.53, best 111.5 vm

files: Lab GPU box, experiments/lunarlander-test-pg-test/sb3-ppo-20261019-153840

Return is climbing

Mean return -173 over the first quarter of episodes, -45.7 over the last.

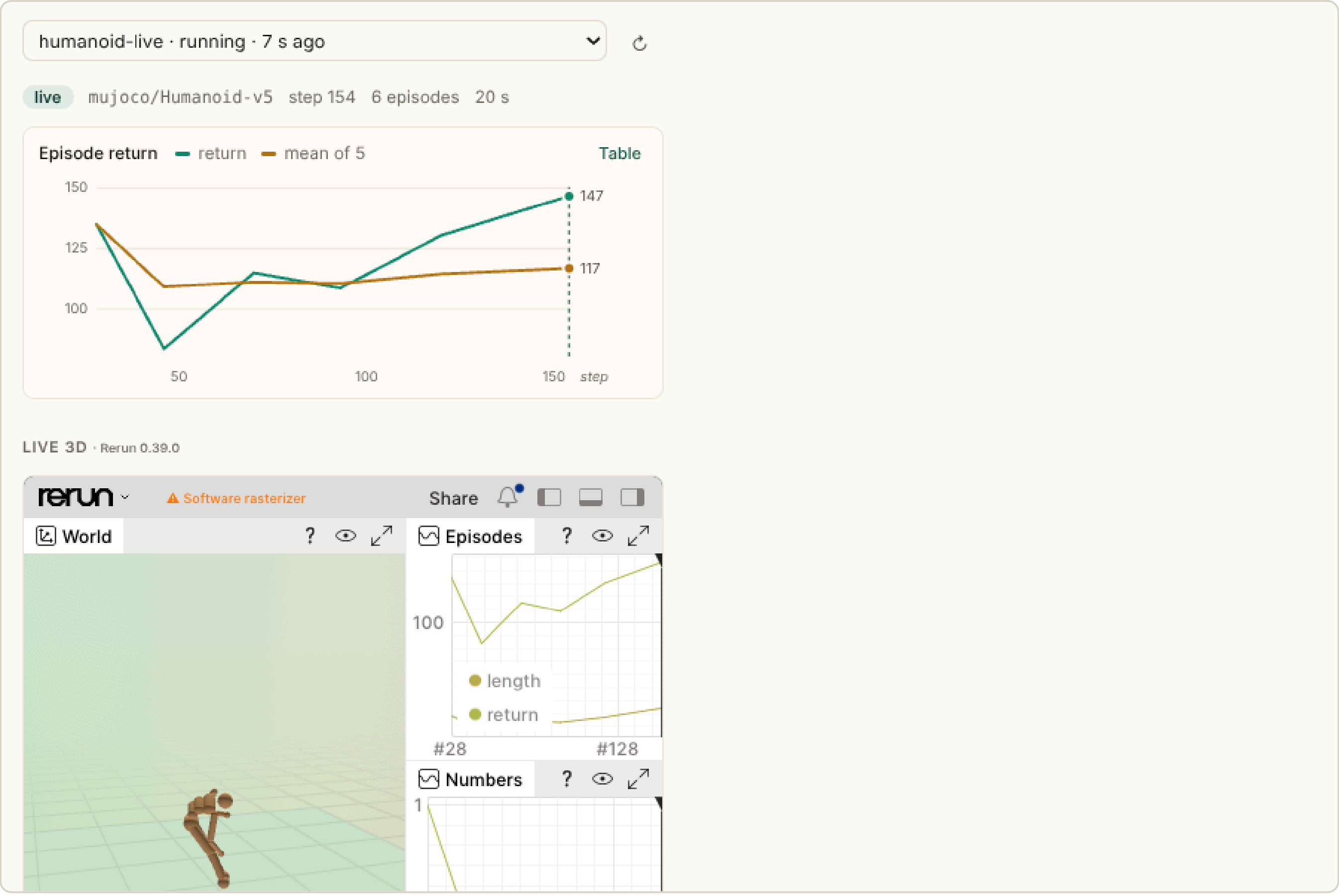
Try: Keep going; compare against the paper's or the env's reported score to know how far there is to go.

train/learning_rate	0.0003
train/entropy_loss	-0.9775
train/policy_gradient_loss	-0.006337
train/value_loss	22.26
train/approx_kl	0.008859
train/clip_fraction	0.06426
train/loss	11.84
train/explained_variance	0.5902
train/n_updates	290
train/clip_range	0.2

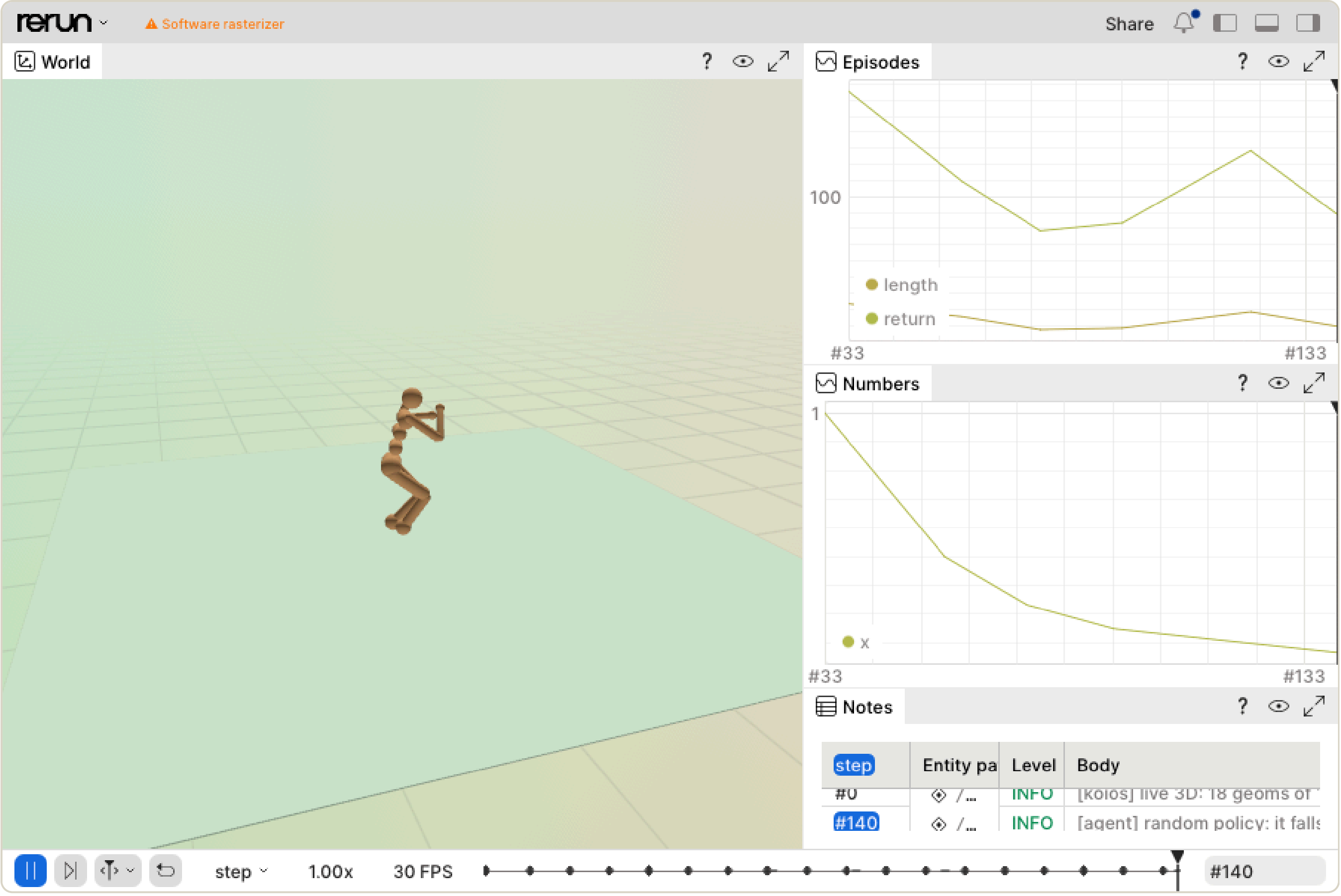
agent step 8.2k clip_fraction 0.12 and value_loss 1154 early: leaving lr at 3e-4, watching for explained_variance to turn positive

media/policy-000000001-0001.gif · media/policy-000015000-0002.gif · media/policy-000030000-0003.gif · media/policy-000045000-0004.gif · media/policy-000060000-0005.gif · events.jsonl · report.md

the Experiments tab: a LunarLander run filed to a server, open — its facts, the finding, the last values, the agent's note, the files, and the buttons for notes and moving


humanoid-live · running · 7 s ago
live
mujoco/Humanoid-v5
step 154
6 episodes
20 s
Episode return
return
mean of 5
Table
150
125
100
147
117
50
100
150
step
LIVE 3D · Rerun 0.39.0
rerun
Software rasterizer
Share
World
Episodes
100
length
return
#28
#128
Numbers
1

Live 3D in the Sim pane, through the Companion the Sim pane on a live Humanoid run: the return chart, then Rerun's viewer embedded with the humanoid on the floor and the episodes and numbers beside it



step	Entity pa	Level	Body
#0	/…	INFO	[koios] live 3D: 18 geoms of
#140	/…	INFO	[agent] random policy: it fall

The viewer in a window of its own Rerun's viewer: the humanoid mid-fall in the World view, episodes, numbers and the notes table on the right, the timeline at the foot

This document records what was built, what was hard about it, and how each difficulty was handled. simulation-process.md is the process: the order of work, every background run and its result, the review against the existing code, and the checklist. It is a log as much as a design. Each entry under Challenges says what is resolved, what is worked around and what is still open.

What is built

PIECE	WHERE	WHAT IT DOES
The telemetry module	`src/sim/koios_telemetry.py`	`Run`: episodes, numbers, videos, dreams, notes, checkpoints, heartbeats, crash capture. CLI: `doctor`, `summary`, `tail`, `note`, `bridge`. One file, no install.
The catalogue	`src/lib/simulators.ts`	31 simulators in 8 families: install lines, GPU needs, Colab or not, example environments, a Gymnasium `env.py` or a native starter for each.
The templates	`src/sim/templates/*.py`	`random_agent.py`, `ppo.py`, `sac.py`, `sb3.py`, `world_model.py`: real files, run by hand on CartPole, InvertedPendulum and LunarLander (see Verified).

PIECE	WHERE	WHAT IT DOES
Project assembly	`src/lib/simProject.ts`	`env.py`, the template, `setup.sh`, `README.md`, `AGENTS.md` (+ `CLAUDE.md`), `LEARNING.md`, the telemetry module; DreamerV3, TD-MPC2 and Isaac Lab run scripts with their logs bridged.
The picker	`src/components/SimPicker.tsx`	**A simulation** on the Playground's home: filter by family, "Dexterous hands", "Runs on Colab", search; pick an environment and a template.
The Sim pane	`src/components/SimPane.tsx`, `src/lib/simTelemetry.ts`	The right-hand pane's **Sim** tab: run picker, status, findings, return and success charts, videos, dreams, grouped numbers with notes as dashed marks, timeline, traceback, config, checkpoints, and the machine's doctor.
Sync fix	`src/lib/playground.ts` (`isBinaryPath`)	The sync carries text; binary files (videos, images, weights) are now left where they are instead of being copied as broken text.
Tests	`scripts/simulation.test.mjs`	Catalogue integrity, every generated `.py` parses, parsing and diagnostics, and the Python module run for real: its events read by the pane's parser, its findings equal to the pane's.

The simulators

FAMILY	SIMULATORS
Classic control	Gymnasium classic control, Box2D, gymnax (JAX)
Physics & locomotion	MuJoCo (Gymnasium), DeepMind Control Suite, MuJoCo Playground (MJX), Brax, PyBullet, Drake
Arms & hands	Gymnasium-Robotics (Shadow hand, Adroit, Fetch), MyoSuite, Meta-World, robosuite, ManiSkill 3, Genesis, Isaac Lab
Games & open worlds	Atari (ALE), MiniGrid, Crafter, Craftax, ViZDoom, NetHack (NLE), MineRL
Multi-agent	PettingZoo
Driving	highway-env, MetaDrive, CARLA
Engines & 3D worlds	Unity ML-Agents, Godot RL Agents, Habitat-Sim
Datasets	Minari (D4RL's successor)

Dexterous hands specifically: the Shadow hand and Adroit (Gymnasium-Robotics), MyoSuite's musculoskeletal hand, the LEAP hand (MuJoCo Playground), the D'Claw (ManiSkill), and the Allegro and Shadow hands in Isaac Lab.

Templates, and what each teaches

TEMPLATE	FOR	WHAT TO WATCH
Random agent	every Gymnasium simulator	the floor: what chance scores; does it render here
PPO (one file)	discrete or continuous, flat or pixel observations	return; `approx_kl` < ~0.05; entropy falling slowly; `explained_var` → 1
SAC (one file)	continuous control: arms, hands, MuJoCo	return after the random warm-up; `alpha` falling; `q_mean` against real returns
Stable-Baselines3	a tested baseline; `MultiInputPolicy` for dict observations (goal-conditioned hands)	SB3's own names
World model from scratch	anything that renders, or a built-in ball world	`loss/recon`, `loss/dynamics`, the dream video and its error over the horizon
Starter (native)	simulators with their own API (JAX, engines, servers)	a first recorded rollout; Brax and MuJoCo Playground also train with `--train`
DreamerV3, TD-MPC2	their repositories, as published (DMC, Atari, Crafter; DMC, Meta-World)	their metrics, bridged
Isaac Lab	RSL-RL PPO on an Isaac Lab task, headless	videos bridged; scalars in the TensorBoard pane

---

The telemetry format (schema 1)

A run is a folder, `runs/<name>-<YYYYmmdd-HHMMSS>/`:

- `run.json`: `{schema, id, name, sim, config, started, status, host, argv, pid}`, rewritten at the end with `status` (`finished` / `crashed` / `stopped`), `ended` and `error`.
- `events.jsonl`: one JSON object a line, appended and flushed as it happens. Every event has `kind` and `t` (machine seconds); most have `step` (**environment steps**, so runs compare).
- `media/`: MP4 (H.264, `yuv420p`, `faststart`), else GIF, else `.npz` frames.

`KIND`	FIELDS
`start`	`name`, `sim`, `config`, `host`, `resumed`
`scalars`	`step`, `values: {name: number}`; names group by their prefix (`loss/policy`)
`episode`	`step`, `index`, `return`, `length?`, `success?`, `extra?`
`media`	`step`, `name`, `path` (from the run's folder), `type` (`video`, `gif`, `image`, `frames`), `frames?`, `fps?`, `bytes`, `caption?`
`dream`	`step`, `name`, `path`, `type`, `error: [per-frame MSE]`, `context` (frames given, not predicted), `fps`, `horizon`
`note`	`step`, `text`, `by` (`code`, `agent`, `human`, `koios`), `tag?` (`problem`, `spaces`, `config`...)
`checkpoint`	`step`, `path`, `metrics`
`heartbeat`	every 10 s from a thread: `step`, `env_steps`, `sps`

`KIND`	**FIELDS**
`end`	`status`, `step`, `episodes`, `elapsed`, `error?` (the traceback)

NaN and the infinities are written as the strings `"nan"`, `"inf"`, `"-inf"`: JSON has no NaN, and a NaN loss is exactly what has to be seen.

Challenges, and how they are handled

1. A browser can't reach the machine's ports, and Colab's least of all

Resolved. Nothing is served from the machine. The pane runs a few lines of Python in the runtime's small second kernel (`runQuietly`, the same path the TensorBoard pane takes) that lists the runs and returns the focused run's file **from the byte where the last read stopped**, ending on a whole line. A long run costs only its new lines; a file longer than 6 MB is read from its last part on first sight (and the pane says so). Polls run every 3 s while a run is live, every 15 s otherwise, and at once while there is more of the file to read.

2. The sync carries text up to 2 MB, and videos are binary

Resolved, and a bug fixed on the way. The sync read every file as text, so a `.png` brought back by the default rules (`*.png` is one) arrived corrupted. Binary files are now left where they are (`isBinaryPath`), and the Sim pane fetches media **from the machine, byte for byte, in 2 MB base64 chunks**, played from a blob (up to 64 MB a file; past that it says where the file is). The fetch refuses any path outside the project's folder.

3. No display: simulators that open windows fail on Colab and servers

Resolved, and verified. `koios_telemetry` picks an off-screen renderer **at import**, before any simulator loads: on Linux with no `DISPLAY`, `MUJOCO_GL=egl` when there is a GPU (an `nvidia-smi` or a `/dev/dri` render node), else `osmesa`; pygame gets `SDL_VIDEODRIVER=dummy`. Choosing EGL merely because its library is present was wrong: on a CPU container the library exists and fails (found while testing here). Every generated `env.py` and starter imports the telemetry first, and a test checks that order. `run.grab(render, frames)` and `run.wrap` turn a renderer that fails into one note on the timeline ("rendering failed, so no videos… run the doctor"), and the run goes on training. The doctor renders a small MuJoCo scene in a subprocess to prove it works.

4. Which simulator runs where

Resolved for what can be known in advance. Each catalogue entry says `gpu: no | helps | needs | rtx`, `colab`, `linux`, `video`. The picker shows it on each card and filters by "Runs on Colab"; the place step warns when an RTX-only or GPU-only simulator is about to go to Colab (`placeWarning`). On the machine, `python koios_telemetry.py doctor` (and the pane's "This machine" section) reports GPUs, EGL / OSMesa / Vulkan, ffmpeg, Java, torch and JAX, and which of the 31 simulators import.

5. Isaac Lab needs an NVIDIA RTX GPU, never Colab

Worked around. It runs on a machine added as a Jupyter server (a lab box, a rented RTX GPU) through the existing Companion / server path. Its own training script runs headless with `--video`; its videos are **bridged** into the Sim pane as they are written and its scalars, which RSL-RL writes as TensorBoard files, show in the TensorBoard pane. Isaac Sim's install (Python 3.11, NVIDIA's package index) is left to its guide, linked from the project: pinning it here would go stale within a release. *Open:* starting and stopping rented GPUs from the page, which the Playground's own design lists as not built.

6. Existing codebases log their own way

Resolved for the common formats. `python koios_telemetry.py bridge <metrics.jsonl | .csv | glob> [--videos DIR] [--follow | --until-pid PID]` follows another codebase's log as it grows and writes a Koios run: step keys (`step`, `global_step`, `_step`, `frame`…), episode keys (`episode/score`, `episode_reward`, `eval/episode_reward`…), lengths and success; videos are copied in when they stop growing. DreamerV3 (`metrics.jsonl`), TD-MPC2 (`eval.csv`), SB3 (`progress.csv`) and Isaac Lab's video folders are covered; the run scripts start training, bridge until its process ends, then close the run. *Open:* a bridge from W&B-only codebases (DIAMOND), which would need W&B's offline files.

7. Torch on a small CPU can be a hundred times slower with more threads

Found while testing, resolved. SAC ran at ~3 steps a second here. Profiling showed every tiny layer taking milliseconds; a benchmark gave **0.26 ms on one thread against 25 ms on four**, the thread pool fighting a container's CPU quota. Colab's two-core CPUs see the same, more mildly. PPO and SAC now use one thread on the CPU (`KOIOS_TORCH_THREADS` to change it); SAC then did 15,000 steps in 2 minutes and solved InvertedPendulum. The doctor measures both on each machine and prints the `export` to use.

8. A shared actor-critic body stopped PPO from learning

Found while testing, resolved. The first PPO shared one MLP body between the policy and the critic. The value loss (returns in the hundreds) dominated the gradient, the whole gradient was clipped to 0.5, and the policy's part shrank to nothing: `approx_kl` ~1e-4, `clipfrac` 0, return flat. **The pane's own diagnostic said "Return isn't improving"** on that run. Flat observations now get separate bodies, as CleanRL does (images keep one shared CNN, as the Atari PPO does); CartPole then climbed from 22 to 262 mean return in 100k steps, best 500.

9. Colab sessions end

Partly resolved by what exists. The `koios` checkpoint helper (`.reader/pylib/koios`) saves on a cadence and on SIGTERM, and the page offers to resume elsewhere. The telemetry records `checkpoint` events, writes a heartbeat every 10 s, and a run that stops writing for two minutes shows as **stalled** with what to try. `Run(resume=dir)` carries on writing to the same run. AGENTS.md tells agents to checkpoint and resume. *Open:* tying `Run(resume)` to `KOIOS_RESUME` automatically.

10. Agents must keep the telemetry and explain themselves

Resolved by the briefing, checked by the pane. Every project's `AGENTS.md` (Codex reads it; `CLAUDE.md` includes it for Claude Code) makes the telemetry a rule: one `Run`, episodes, environment steps on the x axis, videos on a schedule, dreams for world models, **a note for every change with its reason** (`run.note(...)` or `python koios_telemetry.py note runs/<run> "…" --by agent` from a terminal), `LEARNING.md` entries written for a learner, and `python koios_telemetry.py summary` read before changing anything. Agent notes appear as dashed lines on every chart, so "the return jumped here" sits next to "lr 3e-4 → 1e-4: approx_kl was 0.2".

11. Two implementations of the diagnostics

Resolved by a test. The findings exist in Python (for agents, in `summary`) and in TypeScript (for the pane). `scripts/simulation.test.mjs` runs both on the same ten synthetic runs (improving, no learning, collapse, NaN, entropy collapse, exploding value loss, crashed, stalled, world-model drift, world model not improving) and requires the same findings. The rule ids match.

12. Long and noisy runs

Resolved. Scalars are thinned on arrival past 20,000 points a series and drawn downsampled; returns are drawn with a running mean (window n/10, at least 5); videos are recorded one episode in `video_every`, capped at 600 frames and 256 px, so a run's media stays megabytes, not gigabytes.

13. Versions move

Worked around. Gymnasium ids change version (`-v4` to `-v5`); `env.py`'s `resolve()` takes the newest registered version of the name asked for. `make_env` retries without a render mode for environments that don't offer `rgb_array`. Simulators with fragile installs say so on their card (Box2D's swig, MineRL's JDK 8, ML-Agents' Python 3.10, Habitat's conda).

14. A live 3D view, from a browser

Resolved, for a Companion; a file elsewhere. Watching a robot move as it trains, not a clip of it after. The choice was Rerun: its web viewer runs in the page, it shows 3D, images and time series on one timeline, and its Python SDK logs anything. What a run does now (`koios_telemetry`, with `rerun-sdk` installed, which `setup.sh` does): a gRPC server sink and a `scene.rrd` file sink together (a plain `rr.save()` would replace the stream), the web viewer on a port, and **scene.rrd served as a stream that follows the file** on a third port. `run.scene(env)` logs a MuJoCo model's geoms once — spheres, capsules, boxes, cylinders, ellipsoids, planes, meshes, with their colours — and `run.scene_step()` moves them (every second step, at most thirty times a second); `run.wrap(env)` does both by itself. Scalars, episodes and notes are mirrored, and a default blueprint opens the world large with the series beside it. The Sim pane embeds the viewer in a **Live
3D** section.

Three things were hard:

- **A browser can't reach the machine's ports.** The Companion (0.18.0) gains `/companion/ports/<ticket>/<port>/…` (`PortProxy`): a reverse proxy to a port on its computer, the ticket an HMAC of the port with the token (an iframe carries no Authorization header), answers **streamed chunk by chunk** — the VS Code proxy beside it buffers whole answers, which would never show a frame — and websockets relayed. The page and the Companion compute the ticket the same way; both test suites pin one vector. On a plain Jupyter server or Colab there is no proxy: the pane offers `scene.rrd` to download and open in Rerun on this computer.

- **Rerun's own stream can't be proxied by path.** The viewer takes its gRPC source only as `rerun+http://host:port/proxy`, an origin and nothing else; found when the viewer said *Failed to parse URL* for the proxied address. Instead the run serves its `scene.rrd` over HTTP as a never-ending chunked response that follows the file, which the viewer loads from any URL and decodes as the bytes arrive. The gRPC server stays for the native viewer on the machine itself (`rerun rerun+http://127.0.0.1:<port>/proxy`).
- **MuJoCo's floor.** A 40 m plane drawn whole makes the body a speck in the viewer's first framing; the floor drawn is capped at 6 m across. Rerun's capsules start at the origin where MuJoCo's are centred; they are shifted by half their length.

Verified here: a random Humanoid-v5 run, the viewer loaded in headless Chromium through the real `PortProxy` (a tornado app with the handler and a paired Companion's state), the humanoid moving, the series and notes beside it; and the same inside the Sim pane (the pictures at the top). Not verified: through the Companion's HTTPS tunnel from another network (the proxy is the same code either way), and in the desktop app's Electron window.

15. Experiments, filed as they end, with notes

Built. A run that is over becomes an **experiment**: a record (`src/lib/experiments.ts`: where it ran, how it ended, returns first/last/best, success, the last value of each number, the findings, the agents' notes, the config) kept in the project's **Logbook → Experiments** tab, and a folder with the run's files — `events.jsonl`, the videos, the dreams, `scene.rrd`, `report.md`, `report.json` — in **Google Drive by default**, under the Koios folder's `Experiments/<project>/<run>/`, or on **a computer of yours** (any Jupyter server or Companion the page knows), chosen once on the tab or per experiment with *Move files to…*. The workspace files runs by itself while the machine is connected (`src/components/ExperimentFiler.tsx`, every twenty seconds, runs that ended in the last two days), says so in a toast with the folder and the move, and the Sim pane shows *filed in Google Drive ↗* on the run. A shared project's room keeps the records too (`worker/logbook.js`, `experiment:` keys, cleaned like the rest).

Notes. By default the model the app is set to for Ask AI (Claude, DeepSeek or Gemini, on your keys or the site's) writes the notes from the record and the project's `LEARNING.md`: what was tried, what happened, what it means, what next, and one idea explained for a learner. They are kept in the record and as `summary.md` beside the files. *Ask Claude Code* / *Ask Codex* on an experiment queues the request to that agent on the machine instead, which can read the run's files itself and writes `runs/<run>/summary.md` and a line in `LEARNING.md`.

What was hard: a logbook entry has to stay small (the room keeps each under a Durable Object's value limit), so the record holds digests and the folder holds the files; media are capped (8 MB a file, 40 MB a run) and what is left out is named; a Jupyter server takes bytes through its contents API (`writeBase64`, added to the file hosts), Drive through its own; and a run must be filed once, which this browser remembers (`experiments:filed:<playground>`).

16. A paper's experiments, wired to a simulator by the agents

Built. A project started **from a paper** now carries the simulation toolkit — `koios_telemetry.py` and `koios_simulators.json`, the catalogue as data — and its `AGENTS.md` (read by Codex, by Claude Code through `CLAUDE.md`, and named to the page's own agent) says: if the paper's experiments run in a simulator, `python koios_telemetry.py simulators` lists them with what each needs, `python` `koios_telemetry.py use <simulator> [env]` writes `env.py` and `setup.sh` (and a starter for a simulator with its own API), log every run through `koios_telemetry.Run`, keep the paper's reported numbers in the README. The page writes the same two files to every machine's helper path, so the commands work in any project's console; Claude Code and Codex started from the page get that path in their `PYTHONPATH` and a line in their briefing naming the commands, and `use` run from there copies the module and the catalogue into the folder, so the project stands on its own. A test holds the `env.py` the machine writes equal to the one the page writes. When the files are on a Companion of yours and the code runs elsewhere (Colab, a GPU server), the agent pane also offers the agent **on the files' computer**, editing them where they are and running on the compute with `koios run` (docs/compute-link.md, section 0).

17. Not every browser plays H.264

Worked around. Videos are MP4/H.264, which Chrome, Edge, Safari and Firefox play. Open-source Chromium builds (the one Playwright drives, some Linux packages) ship without H.264: found when screenshotting the pane, where the player loaded the video but stayed at 0:00. `KOIOS_VIDEO=gif` makes the module write GIFs instead, which every browser plays, at a larger size.

Verified

On this repository's Linux container (4 CPUs, no GPU), with `gymnasium 1.x`, `mujoco 3.x`, CPU torch:

- `ppo.py` on CartPole-v1, 100k steps: mean return 22 → 262, best 500, three videos; finding: *Return is climbing*.
- `sac.py` on InvertedPendulum-v5, 15k steps: 2 min 17 s, best return 1000, 23 videos.
- `sb3.py` (PPO) on LunarLander-v3, 20k steps: mean return −155 → −70, videos of the policy.
- `world_model.py` on CartPole and on the built-in ball world: reconstruction and dynamics losses logged, dream videos with per-frame error. On the ball world, after 600 iterations, the error stayed near 6e-3 for the five given frames and rose to 8e-3 by frame 45: compounding error, visible on the horizon chart.
- The Sim pane and the picker, rendered in Chromium against these runs (the kernel call replaced by a local Python): findings, charts, the video list, the dream view and the hands filter all drawn from the real files.
- MuJoCo off-screen rendering through OSMesa to H.264 MP4.
- The doctor, the summary, `note` and `bridge` from the CLI.

Not run here (no GPU, or a server to install): the JAX simulators, ManiSkill, Genesis, Isaac Lab, CARLA, Habitat, MineRL, ML-Agents, Godot. Their starters follow each project's documented API and every generated file is checked to parse; they are the first thing to try on a GPU machine.

Next

- Compare runs: two or more runs' returns on one chart (before and after an agent's change).
- An "Ask the agent about this run" button that sends the summary and the findings to the agent pane.
- `Run(resume=…)` from `KOIOS_RESUME`, so a resumed run continues its own timeline.
- Rented GPUs started from the page (the Playground design's open item), for Isaac Lab and CARLA.

- A W&B offline bridge, for DIAMOND and other W&B-only codebases.

PART 2

How it was built, checked and reviewed

The companion to simulation.md, which describes *what* the simulation projects and the Sim pane are. This one is the process: the order the work went in, every command that ran in the background and what came of it (including the three the task list shows as *Failed*), the review of the change against the code that was already there, and the checklist to run before merging or after changing any of it. Written so that someone who was not in the session can retrace it, repeat it, and trust what it claims.

The branch is `claude/zealous-bardeen-vr8z49`; the first commit is `f941149`, the review fixes are the commit after.

---

1. The ask

Two questions, then a build request:

1. *What do people use for RL and robotics simulation?* Answered in the conversation, then turned into the catalogue (`src/lib/simulators.ts`): 31 simulators, each with its install, what it needs, and its environments.
2. *Can the site host world-model and RL work, with coding agents building it and telemetry so I can see and learn?* Yes; this is the build.
3. *As many simulation options as possible, every recommendation implemented, tools for the limitations, and documentation of what was done, the challenges and their resolution.*

2. The order of work

Each step was chosen so the next could be checked against something real rather than assumed.

#	STEP	WHY IN THIS ORDER
1	Read the Playground: `src/lib/playground.ts`, `PlaygroundWorkspace.tsx`, the Metrics and TensorBoard panes, `telemetry.ts`, `koiosHelper.ts`, `colab.ts`'s `runQuietly` and the Jupyter contents helpers	To add to the app the way it is built: panes read the machine through the second kernel, helpers are written to `.reader/pylib`, the sync carries text.
2	Write `src/sim/koios_telemetry.py` and run it for real against Gymnasium and MuJoCo in a venv	The format had to exist and be proven before anything read it. The first run found the EGL-without-a-GPU problem (section 4).
3	Write the templates as real files (`src/sim/templates/*.py`) and the catalogue; generate projects from them with a small script and run each	Running them found the two learning bugs (PPO's shared body, torch's thread pool) that no amount of reading would have.
4	Write the pane's reader (`simTelemetry.ts`) with the same diagnostics as the Python, then the Sim pane, the picker and the project assembly	Two implementations of one rule set, held equal by a test (section 5).
5	Tests (`scripts/simulation.test.mjs`), `tsc`, the full suite, a production build	Section 5.

#	STEP	WHY IN THIS ORDER
6	A UI harness: the real components in Vite with `runQuietly` swapped for a local Python, screenshotted in Chromium against the real runs	To see the pane and the picker drawn from actual files, not to trust the code. Found the H.264 caveat.
7	Docs, commit, push	
8	Review (this document, section 6): the three *Failed* tasks explained, the diff against existing code re-read, three fixes	

3. The background tasks, one by one

The session's task list shows 26 finished tasks. The ones that mattered, in order, with what each established. Durations are from the list.

TASK	RESULT	WHAT IT ESTABLISHED
Install deps and a Python venv with gymnasium	ok, 18 s	`gymnasium 1.x`, `mujoco 3.x`, `imageio-ffmpeg` in a venv under the scratchpad; `npm ci` for the repo.
Exercise the telemetry module end to end	**crashed by design**: the test script raises at the end	The run recorded its own crash (`status: crashed`, the traceback in the `end` event), three CartPole videos as MP4, a dream with per-frame error, a note; `summary` printed all of it. Also found: `doctor` crashed on `importlib.util.find_spec("google.colab")` when `google` isn't installed. Fixed with `_have()`.
Install OSMesa/EGL and recheck rendering	12 s	With `libegl1` present but no GPU, `MUJOCO_GL=egl` fails inside MuJoCo. The library being there is not enough.
Prefer OSMesa without a GPU and retest MuJoCo video	5 s	The renderer is now EGL only when `nvidia-smi` or a `/dev/dri/renderD*` node exists, else OSMesa. HalfCheetah rendered off-screen to a 64 KB H.264 MP4 (checked with ffmpeg: `h264 (High) yuv420p`).
Install CPU torch and SB3 for template testing	5 m 36 s	CPU torch, SB3, Box2D (via swig).
Run generated PPO project on CartPole	13 s	**Return flat** (22 → 28 over 60k steps). The pane's own diagnostic flagged it: *Return isn't improving*. Cause: one MLP body shared by actor and critic; the value loss dominated the clipped gradient.
Separate actor/critic bodies and rerun PPO	21 s	22 → 262 mean return, best 500, in 100k steps. *Return is climbing*.
Run SAC, SB3 and world-model projects	**Failed (exit 144)**	See section 3.1. SB3 on LunarLander finished inside it: −155 → −70, three videos. SAC was running at ~1 step/s.

TASK	RESULT	WHAT IT ESTABLISHED
Run SAC alone and time it	8 m 21 s	Still ~3 learning steps/s with nothing else running: not contention.
Profile SAC	1 m 43 s	All the time inside `torch._C._nn.linear` and `torch.relu`: 3 ms for a 256-wide layer.
Benchmark torch thread counts	11 s	**0.26 ms on 1 thread, 25.1 ms on 4** for the same layer on this 4-CPU container. The thread pool against a CPU quota.
Single-thread default and doctor thread benchmark	5 s	PPO and SAC set one CPU thread (`KOIOS_TORCH_THREADS` overrides); the doctor measures both and prints the export.
Validate SAC and world model with one thread	**Failed (exit 144)**	See section 3.1. SAC inside it finished: 15,000 steps in 2 m 17 s, best return 1000 (InvertedPendulum solved), 23 videos.
Check existing sync tests and typecheck	33 s	`tsc -b` clean after the sync change.
Run the simulation tests / Fix tests and rerun	14 s / 13 s	First run: 30/32. One test regex was wrong about PettingZoo (it imports its simulator inside `main()`); one needed `external`/`imports` options to load `playground.ts` as the other tests do. Then 32/32.
Picker styles and typecheck	36 s	Clean.
Start harness and screenshot the Sim pane	**Failed**	See section 3.1: Playwright's default browser path, in a throwaway script.
Run harness from inside the repo and screenshot	20 s	Vite needed to run from inside the repo to resolve packages; React needed pre-bundling.
Restart vite with pre-bundled React and screenshot	23 s	The Sim pane drawn from the real SAC run (`docs/sim-pane.png`).
Screenshot the simulator picker	4 s	`docs/sim-picker.png`, on the "Dexterous hands" filter.
Screenshot the world-model run in the pane	13 s	`docs/sim-dream.png`: the dream video, the error over the horizon, the error over training.
Clean up, typecheck and run all tests	1 m 04 s	`tsc` clean; **1736 of 1740 pass; 4 fail in** `glass.test.mjs` **("glass wallpapers")**.

TASK	RESULT	WHAT IT ESTABLISHED
Run the glass tests on the base commit	6 s	The same 4 fail on `main` at `5ce2022`, in a worktree with no change applied. Pre-existing, unrelated.
Production build check	34 s	`vite build` succeeds; the telemetry module's text is in the bundle (the `?raw` import works in Vite as in the test bundler).
Add GIF override, update docs, rerun tests	14 s	`KOIOS_VIDEO=gif`; 32/32.
Add screenshots, commit and push	5 s	`f941149`, pushed.

3.1 The three tasks marked *Failed*

None of the three is a failure of the code in this change. Each is the session's shell wrapper reporting on a long command that was moved to the background and then ended by hand.

- **Run SAC, SB3 and world-model projects** and **Validate SAC and world model with one thread** both exited with code **144**. The commands ran three or two Python jobs in parallel with `&` and `wait`, exceeded the foreground limit, were moved to the background, and were then killed by me (`pkill -f world_model.py`, then `kill` on the remaining jobs) once I had what I needed from them: the first because SAC was crawling and I wanted to profile it alone; the second because the world model on the CPU was at ~1 iteration a second and the runs it had already written were enough to screenshot. Their output files show the jobs that completed (`Done`) and their summaries; the kill produces the 144.
- **Start harness and screenshot the Sim pane** failed in a throwaway Playwright script in the scratchpad: `browserType.launch: Executable doesn't exist at /opt/pw-browsers/chromium_headless_shell-…`. The repo's `playwright` package expects a newer browser build than the one pre-installed in this environment. Fixed in the next task by launching with `executablePath: '/opt/pw-browsers/chromium'`, as the environment's notes say to. Nothing in the repository uses that script.

The world-model run that was killed had, by then, written `loss/recon`, `loss/dynamics`, two dreams and their error curves, which is what `docs/sim-dream.png` shows; `simulation.md` reports those numbers and says the run was not taken further on a CPU.

4. What testing found that reading would not have

FOUND	WHERE	FIX
EGL fails without a GPU even when its library is installed	first MuJoCo render on this container	choose EGL only with a GPU present, else OSMesa
`find_spec("google.colab")` raises when `google` isn't installed	`doctor`	`_have()` guards it
PPO with a shared actor-critic body doesn't learn CartPole	first PPO run; flagged by the pane's own diagnostic	separate bodies for flat observations, shared CNN for images, as CleanRL does

FOUND	WHERE	FIX
torch's default thread pool is ~100× slower than one thread on a small CPU container	SAC at 3 steps/s; profile; benchmark	one thread by default in PPO and SAC; the doctor measures both
The sync copied PNGs (in its default bring-back rules) as text, corrupting them on Colab's kernel path	reading `pullBack` while planning video fetching	binary files left on the machine; the pane fetches them byte for byte
Open-source Chromium can't play H.264	the pane's `<video>` stuck at 0:00 in the screenshot	`KOIOS_VIDEO=gif`
"No episodes logged" fired on a world-model run, which has none by design	first world-model summary	the finding skips runs with dreams
`summary` printed "0 env steps/s" on finished runs	SB3 summary	speed shown only while running

5. Verification matrix

What was run, where, and what passed. "Here" is this repository's cloud container: Linux, 4 CPUs, no GPU, no display, Python 3.13, Node 22.

CHECK	COMMAND	RESULT
Types	`npx tsc -b`	clean
VS Code extension types	`npm run typecheck:vscode`	clean
New tests	`node --test scripts/simulation.test.mjs`	32/32 (catalogue integrity; every generated `.py` parses under Python; event parsing; incremental reads equal whole reads; all ten diagnostic cases; **TypeScript and Python findings identical on the same runs**; the module writing a crashed run, videos, dreams; `bridge` on DreamerV3-style JSONL and TD-MPC2-style CSV; `note` from the CLI; `doctor`; the on-machine listing and chunked media code; the media code refusing a path outside the project; `isBinaryPath`)
Full suite	`npm test`	1736/1740; the 4 failures are `glass.test.mjs` and fail identically on `main`
Production build	`npx vite build`	succeeds; the Python module text is in `assets/index-*.js`
Worker	`npm run check:worker` (wrangler dry run)	succeeds (the Worker is untouched)
CI	`.github/workflows/deploy-site.yml` runs `npm ci && npm run build:pages` on pull requests; `deploy-worker.yml` runs seven unrelated test files	both exercised locally above

CHECK	COMMAND	RESULT
PPO template	CartPole-v1, 100k steps, 1 thread	22 → 262 mean return, best 500
SAC template	InvertedPendulum-v5, 15k steps, 1 thread	best return 1000 in 2 m 17 s, 23 videos
SB3 template	LunarLander-v3, PPO, 20k steps	−155 → −70 mean return, videos of the policy
World-model template	CartPole and the built-in ball world, CPU	losses and dreams logged; error rising over the horizon
Random-agent template	CartPole, InvertedPendulum, HalfCheetah via `run.wrap`	episodes, videos
Off-screen video	MuJoCo through OSMesa	H.264 MP4, confirmed with ffmpeg
The UI	Sim pane and picker in Chromium against the real runs	`docs/sim-pane.png`, `docs/sim-dream.png`, `docs/sim-picker.png`

Not run here, and why: the JAX simulators (Brax, MuJoCo Playground, Craftax, gymnax), ManiSkill, Genesis, Isaac Lab, CARLA, Habitat, MineRL, Unity ML-Agents and Godot need a GPU, a server, a JDK, conda or an engine binary. Their starters follow each project's documented API and are checked to parse. **They are the first thing to run on a GPU machine**, and `simulation.md` says so.

6. The review: does it break anything that was there?

Every file that existed before and was touched, what changed, and why it is safe. (New files can't break existing behaviour except through these.)

FILE	CHANGE	RISK CONSIDERED	VERDICT
`scripts/bundle.mjs`	an esbuild plugin for `?raw` imports	affects every test's bundling	the full suite ran through it: same results as before, the 4 glass failures pre-existing
`src/components/Charts.tsx`	`LineChart` takes optional `marks`	every existing chart	drawn only when passed; nothing passes it but the Sim pane

FILE	CHANGE	RISK CONSIDERED	VERDICT
`src/lib/playground.ts`	`start` gains `'sim'`; `isBinaryPath`; `pushFolder`, `pullBack`, `pullEdits` skip binary files	records with `start: 'sim'` read by an older build; the sync's behaviour	nothing switches on `start` (grepped); `normalise` keeps the value. The sync carried text: on a Jupyter server a binary read already returned null (400) and was skipped silently; on Colab's kernel path it was decoded as UTF-8 and written back corrupted. Now both skip and report it. **A behaviour change, and a fix.**
`src/components/PlaygroundWorkspace.tsx`	the Sim tab, its glyph and palette entry; `ensureHelper` also writes `koios_telemetry.py` (48 KB) to `.reader/pylib`	one more write per machine per tab before the first command	wrapped in the existing `.catch(() => undefined)`; a failed write leaves `helperOn` unset so it is retried; `.reader/` is in the default ignore rules so it is never synced. The "skipped" wording in the Sync report updated to say "over 2 MB, or not text" (review fix)

FILE	CHANGE	RISK CONSIDERED	VERDICT
`src/components/Playground.tsx`	the seventh start card and `fromSim`	the home's grid is `repeat(3, …)`	seven cards wrap to a third row; no layout change needed
`src/styles.css`	new `.sim-*` classes and `.cell-chart .mark`	collisions	all new names; `.mark` scoped under `.cell-chart`
`package.json`	`simulation.test.mjs` in `test` and `test:unit`	CI	the Python tests skip themselves without `python3`; the numpy ones without numpy; none need a simulator
`README.md`	the Playground section: "Seven ways to start", a *A simulation* subsection		prose only

Review fixes made after the first commit (section 2, step 8):

1. **Notebook cells assumed the project folder.** The generated notebook's cells ran `!bash setup.sh`, but a playground notebook's kernel starts where the machine starts it (Colab's `/content`, a server's home); only the console `cd`s into the folder (`shellCell`). The cells now say so and show a `%cd` first.
2. **The Sync report's "over 2 MB"** now also names "not text (videos, images, weights)".
3. **The run listing walked the whole project tree** every 3–15 s with `glob('**/events.jsonl')`, including `node_modules` and `data/`. It is now an `os.walk` that prunes those folders and stops six levels down; the Python `find_runs` does the same, so `summary` and the pane see the same runs. A run under `data/` is not found, by design.

6.1 A second look, with a game and a made-up world

After the review, two more runs were made specifically to see the pane as someone training a game-playing agent or dreaming a world would see it: SB3's PPO on LunarLander (a Box2D game) for 60k steps, and the world-model template on its built-in ball world, both with `KOIOS_VIDEO=gif` so the screenshot browser could play them, and with a note added from the terminal as an agent would (`koios_telemetry.py note … --by agent`). The live screenshot found one more thing:

- **Series of one prefix on one chart only when their scales are close.** SB3 logs everything under `train/`, so `train/value_loss` (~1000) and `train/learning_rate` (0.0003) shared a chart on which only the big one could be read, and the four-name legend overflowed the pane. `scalarCharts` now lets a prefix's names share a chart only while their largest values are within 20× of each other (a test covers SB3's names), and the legend wraps. The screenshots in `simulation.md` were retaken after the fix.

6.2 Live 3D

Asked for after the first review: a live, orbitable 3D view of the simulation rather than clips. Built on Rerun, in this order, each step checked before the next:

1. A smoke test of the SDK here: log a box and a ball, serve, load the web viewer in headless Chromium with a software rasterizer. Found that `rr.save()` replaces the stream sink, then that `set_sinks` with `serve_grpc`'s server does the same: the pairing that works is `GrpcServerSink` + `FileSink` in one `set_sinks`.
2. `koios_telemetry`: the sinks, the viewer, the `live3d` event, scalars/episodes/notes mirrored, `scene()` for MuJoCo geoms, `scene_step()`. A Humanoid-v5 run showed a speck: MuJoCo's 40 m floor; capped, and a default blueprint added.
3. The Companion's `PortProxy` (0.18.0), with a unit test that proves answers stream (three chunks 50 ms apart must arrive as three), the ticket is the credential, POSTs carry body/path/query, nothing-listening is a 502, websockets relay. The first run of the POST test failed in the test's own upstream handler, not the proxy.
4. The end-to-end chain through the real handler: the viewer page came through; the stream didn't — Rerun's URL parser refuses a path before `/proxy`. Resolved by streaming `scene.rrd` over HTTP from the run (`_RrdTail`), which the viewer loads from any URL. Then the humanoid moved live through the proxy.
5. The Sim pane's Live 3D section, with the harness feeding it a paired-Companion server pointed at the proxy: `docs/sim-live3d.png`. The first screenshot showed the Colab fallback instead: Vite caches files under `node_modules` as immutable, where the harness lived, so the edited page never reached the browser until the dev server restarted with `--force`.

Three background commands in this phase ended with code 144 again: each was a long command whose Python child (a run holding ports, a proxy) I ended by hand once it had served its purpose; the `pkill -f` pattern also matched the shell running it twice, which is why the pattern changed to `[r]r-smoke`.

6.3 Experiments and the paper toolkit

Asked for next: results logged automatically where they are accessible, Drive by default, moved elsewhere on request, with notes by the app's model or the signed-in coding agent; and a paper's reimplementation able to reach the simulators through the agents. Built in this order:

1. The record (`src/lib/experiments.ts`) and the logbook's new field, the Worker's cleaning and ops, the store's ops and broadcast handling: pure first, with `scripts/experiments.test.mjs` pinning the digest (its own first numbers were wrong: the first tenth of 0, 2, 4... is 5), the caps, the Markdown, the Worker's refusals, and the room keeping and removing records. Two assertions in `project-room.test.mjs` compared an empty logbook's exact shape and gained `experiments: []`; nothing else in the suite changed.
2. The filing (`src/lib/experimentFiling.ts`): the whole `events.jsonl` read in pieces, media fetched within the caps and written as bytes (`writeBase64` added to the Jupyter and kernel hosts; Drive had it), the record kept and remembered; the notes through `streamModel`; moving between places.
3. The tab, the auto-filer with its toast, the Sim pane's filed line.
4. The catalogue as JSON on the machine and `koios_telemetry.py use` / `simulators`, tested against the page's own `envPy` byte for byte, and run for real: `use gymnasium CartPole-v1` on a bare folder, then the random agent on it.

5. End to end here, with a Jupyter server started in the container as "a computer of yours" and the harness calling the real `fileRun` on the finished LunarLander run: the files (events, five GIFs, report.md, report.json) landed under `experiments/…` on that server and the Experiments tab drew the record (`docs/experiments-tab.png`). Drive itself cannot be reached from here (no Google sign-in), so the Drive place is exercised only by the same code path's Jupyter twin; and no model key is available here, so the notes were not generated in this session.

7. Checklist: before merging, or after changing any of it

```
npx tsc -b && npm run typecheck:vscode
node --test scripts/simulation.test.mjs scripts/experiments.test.mjs   # needs python3 for
some
npm test                                         # expect only glass.test.mjs's 4 pre-
existing failures
npx vite build --outDir /tmp/build && grep -l "koios_telemetry: what a simulation run"
/tmp/build/assets/*.js
(cd companion && python -m unittest $(ls tests/test_*.py | sed 's#/#.#; s#\.py$##'))   # the
Companion: 157 tests, its port proxy's included
```

And on a machine with Python (a venv is enough), the thing itself:

```
python -m venv v && . v/bin/activate && pip install "gymnasium[classic-control]" torch
imageio imageio-ffmpeg
node -e "..."   # or: start a simulation project in the app, Gymnasium / CartPole-v1 / PPO,
and copy its files out
python koios_telemetry.py doctor               # rendering, ffmpeg, the thread advice
python ppo.py --total-steps 100000             # expect the return to climb past 200
python koios_telemetry.py summary              # expect "Return is climbing"
```

On a GPU machine, additionally: Brax or MuJoCo Playground with `--train`, ManiSkill's starter, and Isaac Lab's `train_isaaclab.sh` on an RTX card. None has run yet.

8. How to extend it

- **A simulator**: one entry in `SIMULATORS` (`src/lib/simulators.ts`): `module` must match the doctor's list in `koios_telemetry.py` (a test enforces it); for a Gymnasium one give `imports`/`make`, for a native one a `native` starter that imports `koios_telemetry` before the simulator (a test checks the order) and logs with `Run`. `caveats` for what will bite.
- **A template**: a file in `src/sim/templates/`, imported `?raw` in `simProject.ts`, an entry in `TEMPLATES`, and `templatesFor` deciding when it is offered. Keep its "what to watch" line in `WATCH` and its primer in `PRIMER`: they go into `README.md` and `LEARNING.md` of every project.
- **A finding**: add the rule to `diagnose` in both `koios_telemetry.py` and `simTelemetry.ts` with the same id, and a synthetic case to `CASES` in the test, which requires both to agree on it.
- **An event kind**: write it in `Run._event`, read it in `read_run` (Python) and `applyEvents` (TypeScript); bump `SCHEMA` only if an old reader would misread it.